МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
(Самарский университет)

Институт информатики и кибернетики
Кафедра технической кибернетики

ЛАБОРАТОРНАЯ РАБОТА № 3

«Потоковая обработка в Apache Flink»

по курсу
Большие данные

Группа 6403
Студент_____ А.А. Кучумов
(*подпись*)

Преподаватель,
Ассистент каф. ТК _____ В.И. Проценко
(*подпись*)

Самара 2025

ЗАДАНИЕ

Выполнить следующие задания из набора заданий репозитория https://github.com/ververica/flink-training-exercises:

1. RideCleanisingExercise.
2. RidesAndFaresExercise.
3. HourlyTipsExerxise.
4. ExpiringStateExercise.

Решения могут быть выполнены на двух языках: Java или Scala. Каждому заданию соответствует .java или .scala файл с шаблоном решения и файл с тестом решения. Тесты расположены в папке test.

ХОД ВЫПОЛНЕНИЯ

В рамках лабораторной работы мне предстояло рассмотреть практические реализации ряда задач потоковой обработки данных в рамках комплекта задач Ververica Flink Training Exercises. Задания выполнялись мною на языке программирования Java.

В первом задании – RideCleansing – требуется сформировать подпоток записей «TaxiRide», для которых как начальная, так и конечная географическая позиция принадлежит заданной области (город Нью-Йорк).

Решение задания было реализовано следующим путем: к потоку, порождаемому источником «TaxiRideSource», применяется оператор «filter». Условие принадлежности точки городу формулируется через вызовы статического метода «GeoUtils.isInNYC» для координат начала и конца поездки. Реализованное решение представлено на рисунке 1.

```
private static class NYCFilter implements FilterFunction<TaxiRide> {
    @Override
    public boolean filter(TaxiRide taxiRide) throws Exception {
      return GeoUtils.isInNYC(taxiRide.startLon, taxiRide.startLat) &&
          GeoUtils.isInNYC(taxiRide.endLon, taxiRide.endLat);
    }
  }
```

Рисунок 1 – Решение первого задания

Второе задание – RidesAndFares – посвящено сопоставлению двух событий. Из первого потока данных «TaxiRide» для каждой поездки со своим идентификатором приходят события с временами начала и конца поездки, а из второго потока данных «TaxiFare» для каждой поездки со своими идентификатором приходят события с оплатой поездки, в котором время совпадает с временем начала поездки. Таким образом задача заключается в том, чтобы сопоставить поездку со оплатой по времени старта поездки.

В решении используется функция объединения двух потоков через «RichCoFlatMapFunction», которая позволяет обрабатывать события из потоков «TaxiRide» и «TaxiFare» независимо, но с доступом к общему состоянию. Для хранения промежуточных данных применяются два состояния:

«ValueState<TaxiRide>» и «ValueState<TaxiFare>». Когда из потока поездок приходит событие начала поездки, функция проверяет, есть ли уже сохранённая информация об оплате с тем же идентификатором. Если соответствующая оплата найдена, формируется результирующая пара и состояние очищается. Если же оплата ещё не поступила, информация о поездке сохраняется в состоянии. Аналогичная логика применяется при поступлении события оплаты: если поездка уже сохранена, происходит объединение и вывод результата, иначе оплата временно сохраняется. Такой подход позволяет корректно сопоставлять события из двух асинхронных потоков, независимо от порядка их поступления, используя механизм управляемого состояния Flink. На рисунке 2 приведен код решения второго задания.

```
    public static class EnrichmentFunction extends RichCoFlatMapFunction<TaxiRide, TaxiFare, Tuple2<TaxiRide, TaxiFare>> {

        private ValueState<TaxiRide> rideState;
        private ValueState<TaxiFare> fareState;

        @Override
        public void open(Configuration config) {
            rideState = getRuntimeContext().getState(new ValueStateDescriptor<>(name: "Saved Ride", typeClass: TaxiRide.class));
            fareState = getRuntimeContext().getState(new ValueStateDescriptor<>(name: "Saved Fare", typeClass: TaxiFare.class));
        }

        @Override
        public void flatMap1(TaxiRide ride, Collector<Tuple2<TaxiRide, TaxiFare>> out) throws Exception {
            if (!ride.isStart) {
                return;
            }
            TaxiFare fare = fareState.value();
            if (fare != null) {
                fareState.clear();
                out.collect(new Tuple2<>(ride, fare));
            } else {
                rideState.update(ride);
            }
        }

        @Override
        public void flatMap2(TaxiFare fare, Collector<Tuple2<TaxiRide, TaxiFare>> out) throws Exception {
            TaxiRide ride = rideState.value();
            if (ride != null) {
                rideState.clear();
                out.collect(new Tuple2<>(ride, fare));
            } else {
                fareState.update(fare);
            }
        }
    }
}
```

Рисунок 2 – Решение второго задания

Третье задание – HourlyTipsExercise – посвящено поиску водителя, получившего наибольшую сумму чаевых за каждый час, на основе потока событий с оплатами поездок. Основная идея заключается в агрегировании

чаевых по часовым интервалам и последующем выборе максимального значения для каждого такого интервала.

В реализации поток событий оплаты сначала разбивается по ключу, которым выступает идентификатор водителя, после чего для каждого водителя формируются часовые окна. Внутри этих окон с помощью функции обработки окна вычисляется суммарное количество чаевых, и на выходе формируются кортежи вида «конец окна – идентификатора водителя – сумма чаевых». Далее полученный поток агрегированных значений обрабатывается повторной оконной функцией без ключа, которая уже среди всех водителей за тот же час выбирает запись с максимальным значением чаевых. На рисунке 3 представлен отрывок кода, решающего третье задание.

```
        // start the data generator
        DataStream<TaxiFare> fares = env.addSource(fareSourceOrTest(new TaxiFareSource(input, maxEventDelay, servingSpeedFactor)));

        DataStream<Tuple3<Long, Long, Float>> hourlyMax = fares
                .keyBy((TaxiFare fare) -> fare.driverId)
                .timeWindow(Time.hours(hours: 1))
                .process(new AddTips())
                .timeWindowAll(Time.hours(hours: 1))
                .maxBy(positionToMaxBy: 2);

        printOrTest(hourlyMax);

        // execute the transformation pipeline
        env.execute(jobName: "Hourly Tips (java)");
    }

    public static class AddTips extends ProcessWindowFunction<
            TaxiFare, Tuple3<Long, Long, Float>, Long, TimeWindow> {
        @Override
        public void process(Long key, Context context, Iterable<TaxiFare> fares, Collector<Tuple3<Long, Long, Float>> out) throws Exception {
            float sumOfTips = 0F;
            for (TaxiFare f : fares) {
                sumOfTips += f.tip;
            }
            out.collect(new Tuple3<>(context.window().getEnd(), key, sumOfTips));
        }
    }
}
```

Рисунок 3 – Решение третьего задания

Четвёртое задание – ExpiringStateExercise – про обогащение потока поездок данными об оплате с учётом ограниченного времени ожидания соответствующего события. В отличие от простого сопоставления, здесь рассматривается ситуация, когда одно из событий может не прийти вовсе или прийти слишком поздно, поэтому требуется использовать механизм истекающего состояния, чтобы очищать несопоставленные данные и отдельно обрабатывать такие случаи.

В реализации используется класс, позволяющий не только хранить состояние для каждого ключа – идентификатора водителя, но и работать с таймерами во времени событий. Для хранения промежуточных данных применяются два состояния: «ValueState<TaxiRide>» и «ValueState<TaxiFare>». При поступлении события из одного потока выполняется проверка наличия соответствующего события из другого потока. Если пара найдена, данные объединяются, состояние очищается, а ранее зарегистрированный таймер удаляется. Если же соответствия нет, событие сохраняется в состоянии и для него регистрируется таймер на основе времени события. Метод «onTimer» срабатывает, если за отведённое время пара так и не была найдена: в этом случае оставшиеся данные считаются несопоставленными и отправляются в дополнительные каналы. На рисунке 4 приведено решение данной задачи.

```
public static class EnrichmentFunction extends KeyedCoProcessFunction<Long, TaxiRide, TaxiFare, Tuple2<TaxiRide, TaxiFare>> {

    private ValueState<TaxiRide> rideState;
    private ValueState<TaxiFare> fareState;

    @Override
    public void open(Configuration config) {
        rideState = getRuntimeContext().getState(new ValueStateDescriptor<>(name: "saved ride", typeClass: TaxiRide.class));
        fareState = getRuntimeContext().getState(new ValueStateDescriptor<>(name: "saved fare", typeClass: TaxiFare.class));
    }

    @Override
    public void onTimer(long timestamp, OnTimerContext ctx, Collector<Tuple2<TaxiRide, TaxiFare>> out) throws Exception {
        if (fareState.value() != null) {
            ctx.output(unmatchedFares, fareState.value());
            fareState.clear();
        }
        if (rideState.value() != null) {
            ctx.output(unmatchedRides, rideState.value());
            rideState.clear();
        }
    }

    @Override
    public void processElement1(TaxiRide ride, Context context, Collector<Tuple2<TaxiRide, TaxiFare>> out) throws Exception {
        TaxiFare fare = fareState.value();
        if (fare != null) {
            fareState.clear();
            context.timerService().deleteEventTimeTimer(fare.getEventTime());
            out.collect(new Tuple2<>(ride, fare));
        } else {
            rideState.update(ride);
            context.timerService().registerEventTimeTimer(ride.getEventTime());
        }
    }

    @Override
    public void processElement2(TaxiFare fare, Context context, Collector<Tuple2<TaxiRide, TaxiFare>> out) throws Exception {
        TaxiRide ride = rideState.value();
        if (ride != null) {
            rideState.clear();
            context.timerService().deleteEventTimeTimer(ride.getEventTime());
            out.collect(new Tuple2<>(ride, fare));
        } else {
            fareState.update(fare);
            context.timerService().registerEventTimeTimer(fare.getEventTime());
        }
    }
}
```

Рисунок 4 – Решение четвертого задания

На рисунке 5 показано успешное прохождение тестов для выполненных заданий.

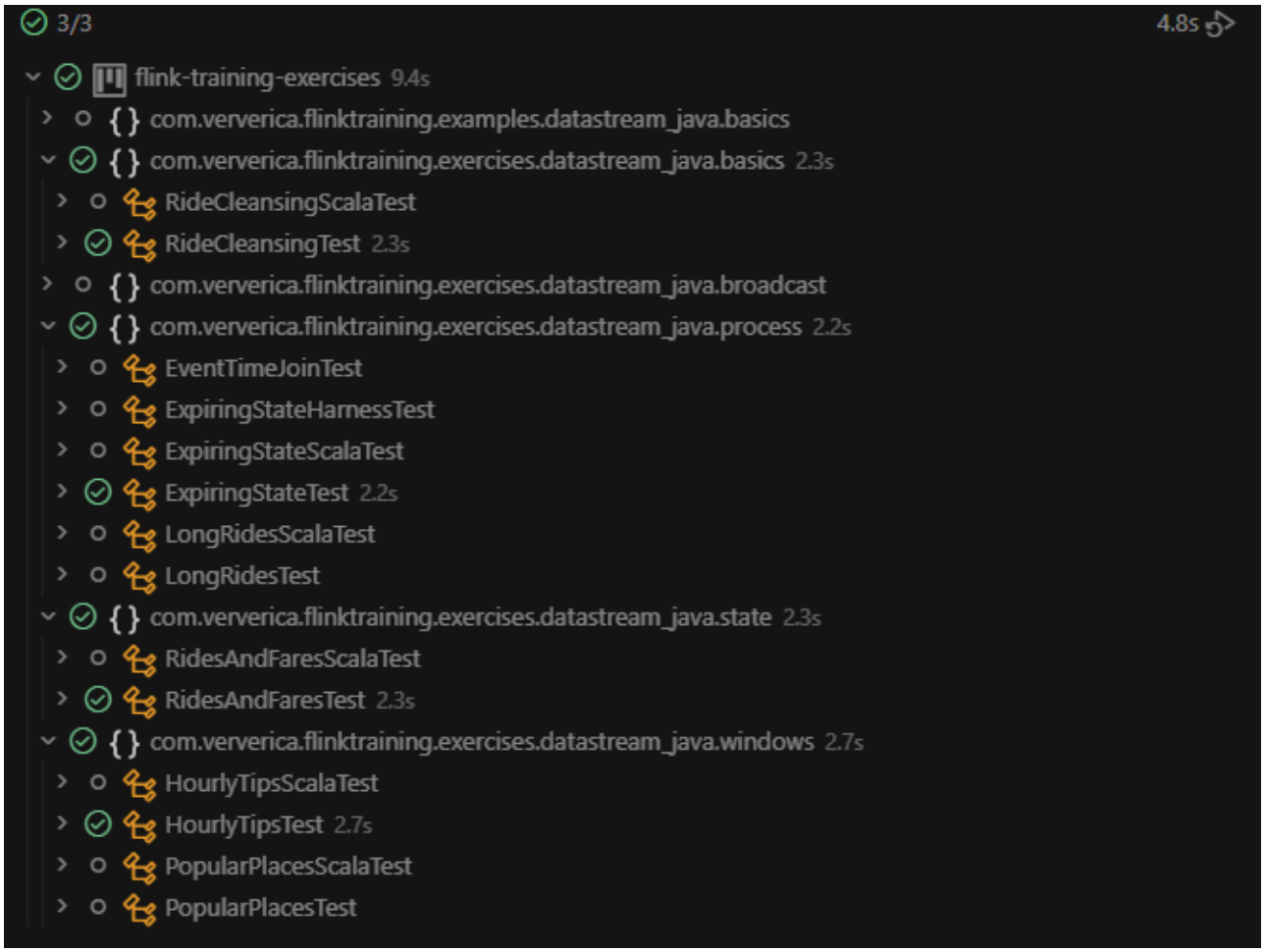

Рисунок 5 – Успешно пройденные тесты

Код решений и тесты на Java представлены в репозитории: https://github.com/artmkchmv/BigData/tree/main/Lab3

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы мною были изучены и реализованы основные подходы к потоковой обработке данных на основе фреймворка Apache Flink. В процессе решения заданий были рассмотрены ключевые механизмы работы с потоками: фильтрация данных, объединение нескольких потоков, оконная агрегация, а также управление состоянием и временем событий.

Практическая реализация задач позволила мне закрепить навыки использования различных типов операторов и функций обработки, таких как «filter», «RichCoFlatMapFunction» и «KeyedCoProcessFunction», а также работы с состоянием и таймерами. Особое внимание было уделено обработке данных с учетом времени событий, что является важным аспектом при работе с распределёнными потоковыми системами, где события могут приходить с задержками и не по порядку.

Таким образом, в рамках работы были получены практические навыки построения потоковых приложений, обработки асинхронных данных и управления состоянием. Корректность выполнения заданий подтверждается успешным прохождением тестов.